

Doc2Vec

문서 임베딩의 이해와 실습

단어를 넘어 문장과 문서의 의미를 벡터 공간에 매핑하다

Word2Vec 리뷰

PV-DM

PV-DBOW

DART 실습

유사기업 검색

TaggedDocument

Gensim Doc2Vec

Presentation

Index

01

Word2Vec 리뷰 & 시각화

- TensorFlow Embedding Projector
- t-SNE / PCA 차원 축소
- Word2Vec의 한계

03

DART 실습 — 유사기업 검색

- 공시 데이터 로드 & 형태소 분석
- TaggedDocument & 모델 학습
- most_similar 결과 확인

02

Doc2Vec 아키텍처

- PV-DM (Distributed Memory)
- PV-DBOW (Distributed Bag of Words)
- 성능 비교 및 Concatenation 전략

04

정리 & Q&A

- Doc2Vec 응용 분야
- 다음 주차 예고 (Transformer)
- Q&A

01

Word2Vec



Word2Vec

리뷰 & 시각화

■ TensorFlow Embedding Projector를 활용한 시각화

gensim.scripts.word2vec2tensor 모듈로 임베딩 파일을 Tensor / Metadata TSV로 변환 후 projector.tensorflow.org에서 고차원 벡터를 3D로 시각화합니다.

```
from gensim.scripts.word2vec2tensor import word2vec2tensor

# Word2Vec 모델 → tensor/metadata TSV 추출
word2vec2tensor("wordvec.bin", "w2v_tensor")

# 생성 파일: w2v_tensor.tsv (벡터값), w2v_tensor_metadata.tsv (단어 목록)
```

업로드 방법

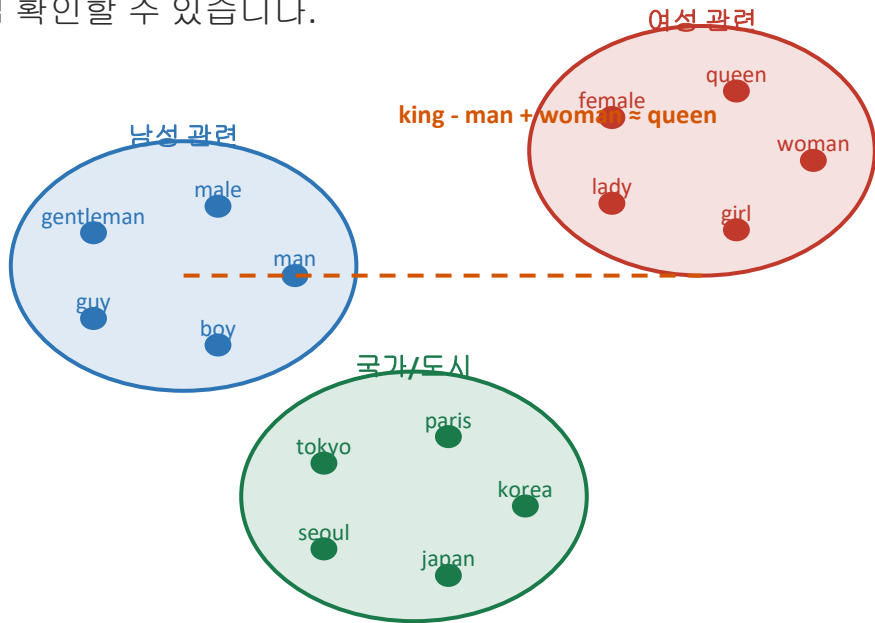
- projector.tensorflow.org 접속
- Load → Vectors 탭 → TSV 업로드
- Metadata 탭 → 메타데이터 TSV 업로드
- 3D 공간에서 단어 벡터 확인

확인 가능한 것들

- PCA / t-SNE 차원 축소 선택
- 비슷한 단어들의 군집(Cluster) 시각적 확인
- king → queen 등 벡터 연산 검증
- 이웃 단어 검색(Nearest Neighbor) 인터랙티브 탐색

■ PCA와 t-SNE로 확인하는 단어 군집

고차원 임베딩 공간을 2D/3D로 압축했을 때, 의미가 유사한 단어들이 자연스럽게 군집을 형성할 수 있습니다.



t-SNE vs PCA

PCA

선형 변환으로 분산을 최대화하는 방향으로 차원 압축. 빠르지만 비선형 구조 포착 한계.

t-SNE

비선형 방법. 고차원 데이터의 지역적 구조를 2D/3D로 압축. 군집 시각화에 탁월.

☞ 의미 유사 단어 → 벡터 공간 근접
→ Word2Vec 분포 가설의 시각적 검증!

■ Word2Vec은 '단어(Word)'를 임베딩한다

실제 NLP 문제는 단어 단위가 아닌 문장·문서 단위입니다. Word2Vec을 문서에 그대로 쓰려면?

② 단순 합산(Average/Sum)

단어 벡터를 모두 더하거나 평균
→ 단어 순서 정보 완전 소실
예) '개가 사람을 물었다' vs
'사람이 개를 물었다' 구별 불가

② Concatenation

모든 단어 벡터를 이어 붙임
→ 문서마다 길이가 달라
고정 크기 입력 불가
실용적 사용이 매우 어려움

② 문맥(Context) 손실

문서 전체의 고유한 주제, 톤,
스타일을 벡터 하나로
표현하기 불가능
→ 문서 분류·추천 성능 한계

→ 해결책: 문서(Paragraph) 자체를 하나의 벡터로 직접 학습하는 Doc2Vec (Paragraph Vector) 등장!

Doc2Vec

02



Doc2Vec
아키텍처

■ 논문: Distributed Representations of Sentences and Documents

2014년 Google 연구진이 발표. 문장·문단·문서를 고정 길이의 밀집 벡터(Dense Vector)로 임베딩하는 비지도 학습 프레임워크.

언어 모델(Language Model)의 기본 원리

K개의 이전 단어가 주어졌을 때 다음 단어를 예측하는 확률 모델

예) "The cat sat" → "on" 예측

$$P(w_t \mid w_{t-k}, \dots, w_{t-1}) = e^{y(w_t)} / \sum_i e^{y_i}$$

PV-DM

Word2Vec CBOW 기반

주변 단어 + 문서 ID → 타깃 단어 예측
문서 ID가 '메모리' 역할

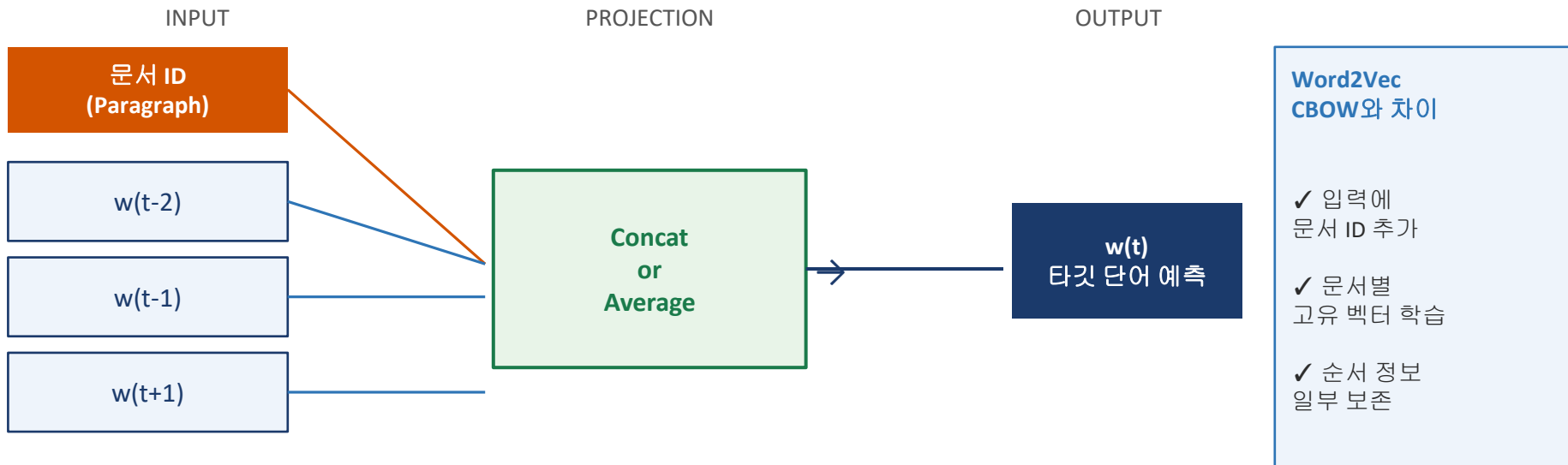
PV-DBOW

Word2Vec Skip-gram 기반

문서 ID 하나 → 문서 내 여러 단어 예측
간단하지만 빠른 학습

■ 핵심 아이디어: 문서 ID를 '메모리(Memory)'처럼 추가

Word2Vec CBOW와 동일하지만, 입력층에 단어 벡터 외에 해당 문서의 고유 ID 벡터를 추가로 입력합니다.



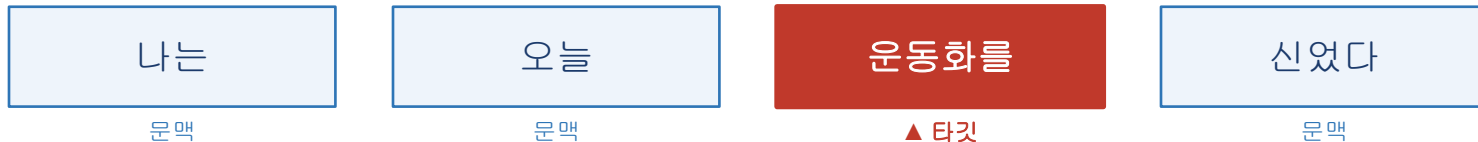
📖 문서 ID의 역할

- 문서 ID는 해당 문서 전체에서 공유되는 '메모리 토큰' 역할
- 같은 문서의 모든 학습 단계에서 동일한 문서 ID 벡터가 업데이트됨 → 문서 전체 주제 학습

예시 문장: "나는 오늘 운동화를 신었다" (문서 ID: D001)

문서 ID (태그)	문맥 단어 (Context)	타깃 단어 (Target)
D001	[나는, 오늘]	운동화를
D001	[나는, 오늘, 운동화를]	신었다
D001	[오늘, 운동화를, 신었다]	(다음 단어)

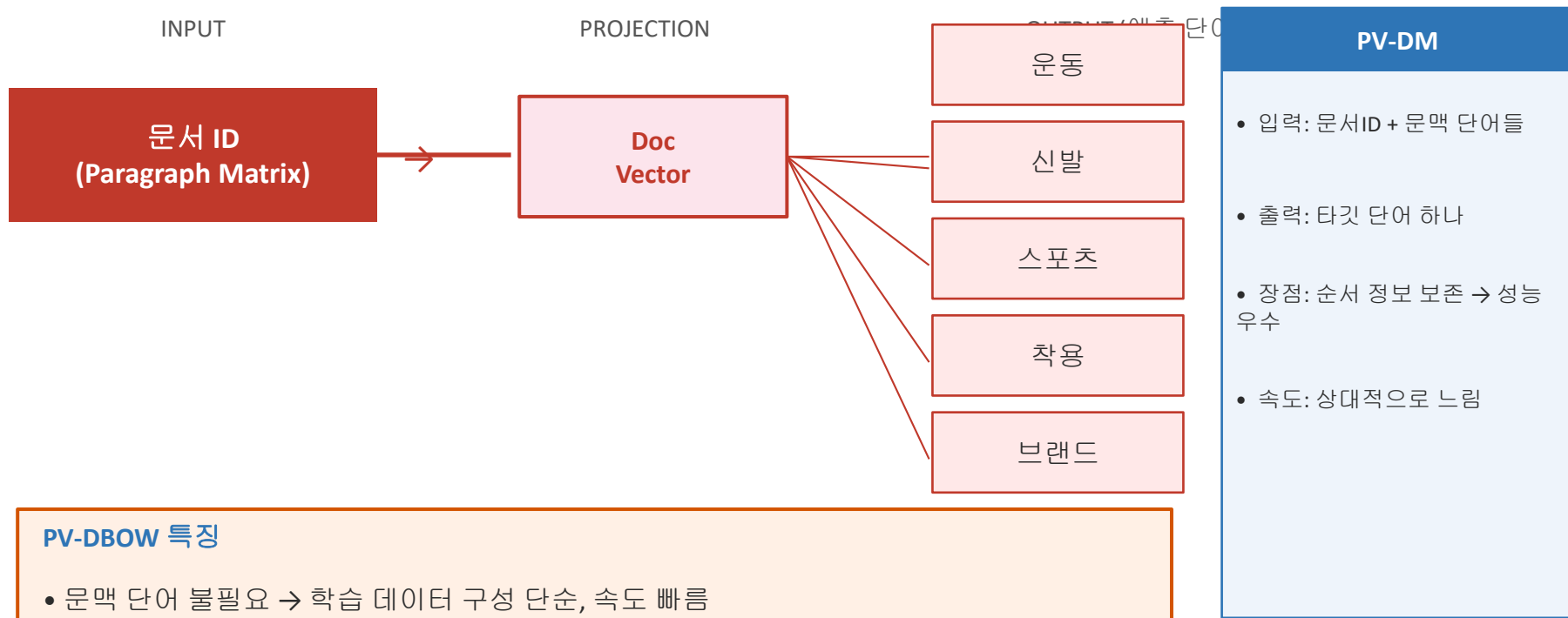
슬라이딩 윈도우 동작 방식 (window = 2)



+ 문서 ID: D001 (위 모든 학습 단계에서 공통으로 입력 → 점진적 업데이트)

■ 핵심 아이디어: 문서 ID 하나로 문서 내 단어들을 예측

Word2Vec의 Skip-gram과 유사. 문맥 단어 입력 없이 문서 ID(Paragraph Matrix)만 입력하여 해당 문서에 등장하는 단어들을 예측합니다.



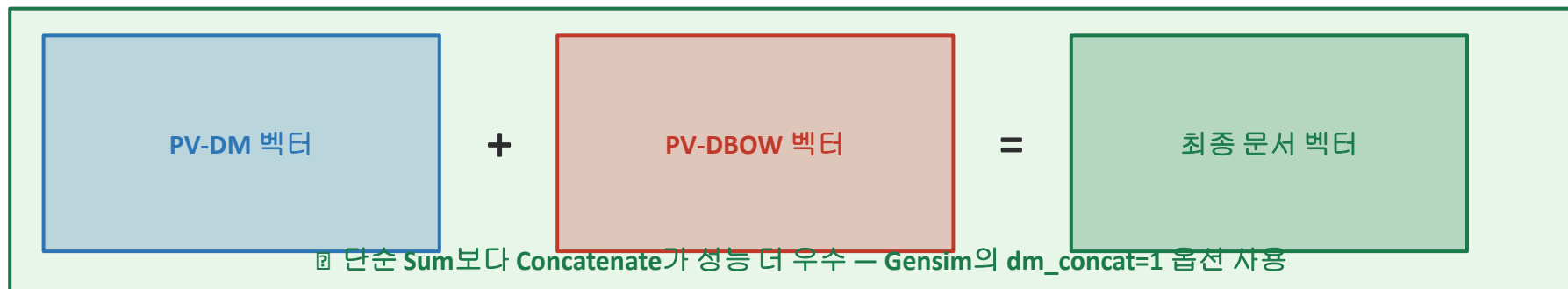
PV-DBOW 특징

- 문맥 단어 불필요 → 학습 데이터 구성 단순, 속도 빠름
- 단점: 문맥 순서 정보 활용 없음 → PV-DM 대비 성능 소폭 낮음

■ 논문 실험 결과 (Google, Mikolov 2014)

모델	문맥 순서 보존	학습 속도	성능
PV-DM	✓ 보존 (좋음)	중간	★★★★☆
PV-DBOW	✗ 미보존	빠름	★★★★☆☆
PV-DM + PV-DBOW (Concatenate)	✓ 보존	느림	★★★★★

■ 최고 성능: Concatenation 전략



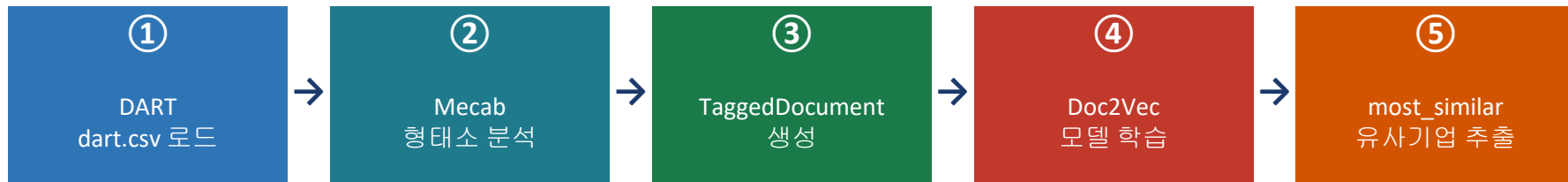
03



DART 실습
유사기업 검색

■ 프로젝트 목표

금융감독원 전자공시시스템(DART) 사업보고서 텍스트 → Doc2Vec 학습 → 비즈니스 모델이 유사한 기업을 AI가 자동 탐색



■ 데이터셋 구조 (dart.csv)

name (기업명)	business (사업의 내용)
동화약품	당사는 종합 의약품 제조 및 판매를 주요 사업으로 영위하고 있으며...
메리츠화재	당사는 손해보험업을 영위하는 보험회사로서 화재보험, 자동차보험...
하이트진로	당사는 맥주, 소주, 위스키 등 주류를 제조하여 국내외 시장에 판매...
LG이노텍	전자부품 제조업으로서 카메라 모듈, 통신용 기판 등을 생산하며...
...(총 2,295 개 기업)	...

■ KoNLPy Mecab 설치 및 형태소 분석

한국어 NLP에서 형태소 분석은 필수. Mecab은 분석 속도가 매우 빠르며 대용량 텍스트 처리에 유리합니다.

```
from konlpy.tag import Mecab

mecab = Mecab()

text = "당사는 종합 의약품 제조 및 판매를 주요 사업으로 영위하고 있으며..."
tokens = mecab.morphs(text)
# → ['당사', '는', '종합', '의약품', '제조', '및', '판매', '를', '...']
```

■ Gensim TaggedDocument 생성

Doc2Vec은 일반 리스트가 아닌 TaggedDocument 객체를 입력으로 받습니다. tags에 기업명을 부여합니다.

```
from gensim.models.doc2vec import TaggedDocument

tagged_corpus_list = []
for i, row in df.iterrows():
    tokens = mecab.morphs(row["business"])
    tagged_corpus_list.append(
        TaggedDocument(words=tokens, tags=[row["name"]]))
# 예: TaggedDocument(words=['당사', '는', '종합', ...], tags=['동화약품'])
```

■ 모델 학습 코드

```
from gensim.models import doc2vec

model = doc2vec.Doc2Vec(
    vector_size=300, # 임베딩 차원 수
    alpha=0.025,    # 초기 학습률
    min_alpha=0.025, # 최소 학습률
    workers=8,      # 병렬 처리 스레드 수
    window=8        # 슬라이딩 윈도우 크기
)

model.build_vocab(tagged_corpus_list) # 단어장 구축
model.train(tagged_corpus_list, total_examples=model.corpus_count, epochs=50)
model.save("dart_doc2vec.model")
```

■ 주요 하이퍼파라미터 설명

파라미터	값	설명
vector_size	300	임베딩 벡터 차원 수. 클수록 표현력 ↑, 메모리 ↑
alpha / min_alpha	0.025	학습률. 에포크가 늘수록 선형 감소시키는 것이 일반적
workers	8	주변 단어 윈도우 크기, 코스를 단편을 문맥 포함

■ most_similar 실행 코드

```
loaded_model = doc2vec.Doc2Vec.load("dart_doc2vec.model")

result = loaded_model.docvecs.most_similar("동화약품")
print(result)
```

■ 출력 결과 (코사인 유사도 기준 Top 5)

```
# model.docvecs.most_similar('동화약품')
[('종근당', 0.9412), ('삼일제약', 0.9387), ('일양약품', 0.9341),
 ('영진약품', 0.9298), ('유한양행', 0.9255), ...]
```

■ 제약업계 군집 형성



모델 해석

단순 키워드('약'이라는 단어 빈도)가 아닌
사업보고서 전체 문맥—제약 산업 비즈니스 구조,
주요 고객사, 판매 채널 등—을 학습한 결과로
동종 업계 기업들이 자연스럽게 군집화됩니다.

most_similar('하이트진로')

```
[('무학', 0.941),
 ('보해양조', 0.938),
 ('국순당', 0.932),
 ('롯데칠성', 0.921), ...]
```

주류업체 군집

주류 산업 군집

하이트진로

무학

보해양조

국순당

롯데칠성

most_similar('LG이노텍')

```
[('LG전자', 0.956),
 ('삼성전기', 0.948),
 ('LG디스플레이', 0.941),
 ('서울반도체', 0.929), ...]
```

전자부품 제조업 군집

전자부품 산업 군집

LG이노텍

LG전자

삼성전기

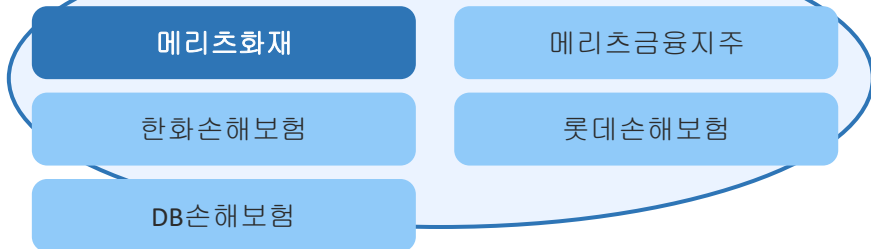
LG디스플레이

서울반도체

`most_similar('메리츠화재')`

```
[('메리츠금융지주', 0.961),
 ('한화손해보험', 0.957),
 ('롯데손해보험', 0.943),
 ('DB손해보험', 0.938), ...]
```

④ 금융/보험 산업 군집



■ 실습 결과 총정리

쿼리 기업	대표 유사 기업	산업 군집
동화약품	종근당, 일양약품	제약
하이트진로	무학, 보해양조	주류
LG이노텍	LG전자, 삼성전기	전자부품
메리츠화재	한화손해보험, DB손해	금융/보험

④ Doc2Vec 실습에서 얻은 인사이트

- 사업보고서 '전체 문서'를 하나의 벡터로 압축 → 단순 키워드 매칭 대비 훨씬 정교한 유사도 계산
- 같은 산업군 기업들이 명시적 분류 없이 자동 군집화 → 비지도 학습(Unsupervised)의 힘
- 코사인 유사도 0.9 이상 → 의미적으로 매우 가까운 사업 구조를 가진 기업 발굴

Supa

04



정리 & Q&A

01

Word2Vec 한계 극복

단어 벡터 평균/합산의 순서 정보 손실
→ 문서 ID 벡터로 문서 전체 의미 학습

02

PV-DM

문서 ID + 문맥 단어 → 타깃 예측
순서 보존, 성능 우수 (CBOW 확장)

03

PV-DBOW

문서 ID 단독 → 여러 단어 예측
단순·빠름, Skip-gram 확장

04

Concatenation 전략

PV-DM + PV-DBOW 결합 사용 시
최고 성능 달성 (dm_concat=1)

■ Doc2Vec 실무 응용 분야

?

유사 문서 추천

뉴스, 논문, 상품 설명서 유사도 기반 추천

??

문서 카테고리 분류

사업보고서, 법률 문서 자동 분류

??

스팸/악성 메일 필터링

문서 벡터 기반 이상 탐지

?

정보 검색(IR)

키워드 검색을 넘어 의미 기반 검색



감사합니다.

Q&A

- 김정순 -

Email : js.kim@deu.ac.kr